

BGP prefix hijacking and RPKI

Follow-along: play the attacker, then the defender

Overview

Definition: Autonomous System (AS)

A network under one administrative control, identified by a number (here AS1 through AS6). The internet is a mesh of ASes; each one originates the address ranges it holds, announcing them to its neighbours so the rest of the internet learns how to reach it.

Definition: BGP and the AS-path

The Border Gateway Protocol is how neighbouring ASes tell each other which address ranges they can reach. Each announcement carries an *AS-path*: the list of ASes the route has passed through. An AS prepends itself before passing an announcement on, so the path both records the route and stops loops.

Definition: Prefix and origin AS

A prefix is an address block written in CIDR, such as 1.0.0.0/22. The origin AS is the one at the end of the AS-path: the AS that first originated the prefix and claims to hold it. For example, in the AS-path 3 2 1 (read left to right, from the AS reporting the route back to the origin), AS1 is the origin: it injected the prefix into BGP, and AS3 and AS2 only forwarded that announcement on their way to you. Nothing in plain BGP checks the origin's claim to the prefix.

Definition: Route

A route is one candidate path to a prefix that a router has heard about: the prefix, paired with the AS-path and the other attributes that describe how to reach it that way. A router that hears about the same prefix from more than one neighbour holds several routes to it and has to choose one, its *best* route, to actually forward traffic on.

BGP runs on trust. When an AS **originates** a **prefix**, its neighbours believe the claim and pass the announcement on, each adding itself to the **AS-path**; nothing forces the originating AS to actually hold what it claims. Two separate mechanisms decide which **route** of several candidates a router ends up using. The first happens in the forwarding table, not in BGP itself: a more specific prefix always beats a less specific one covering the same addresses, regardless of either route's AS-path. The second is BGP's own best-path selection, which chooses between routes to the exact same prefix and runs through several tie-breakers, in order, before it ever looks at AS-path length. This lab's setup (below) turns all of those earlier tie-breakers off, so with everything ahead of it tied, AS-path length becomes the one that actually decides a route here, and the shorter path wins. Neither mechanism consults who is entitled to originate the prefix, so an AS that originates a prefix it does not hold may be able to collect some of the traffic meant for the real holder; how much depends on how many other ASes end up preferring its route, which is exactly what this lab measures.

This lab runs that attack on a six-AS network and then tries to stop it twice. The first defence is the one operators reach for first: the victim splits its prefix into smaller pieces so its own, more specific announcements win longest-prefix match instead of losing on AS-path length, for example originating 1.0.0.0/23 and 1.0.2.0/23 instead of 1.0.0.0/22. You will watch the

attacker match every split until both sides hit /24, the smallest prefix the routing table accepts, and the attacker still holds part of the network. The second defence changes what routers check rather than what either side announces: the victim signs a statement naming itself as the only AS allowed to originate 1.0.0.0/22, and a router that checks a route against that statement refuses one that contradicts it, no matter how specific the contradicting route is.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab    # first time
cd ~/MiniLab && git pull                                       # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

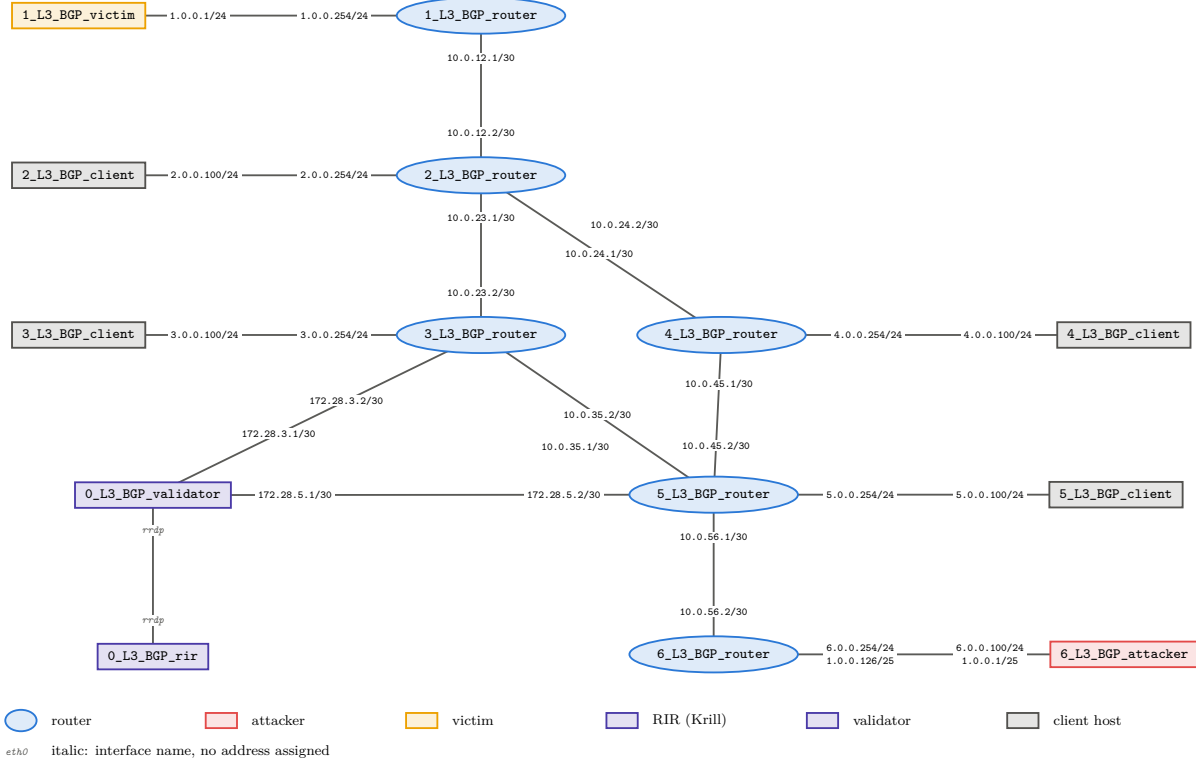
The root menu lists one category per row. Select **Layer 3 and 4 attacks and mitigations**, then **BGP prefix hijacking and RPKI**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring, including the extra containers Part 2 needs later. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: which AS each router currently treats as the origin of the victim's prefix, and what each client's `curl` to the victim's address reads back. Once Part 2 is under way it also reports the RPKI state. Read it before and after each step to see what changed. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node (`as1...as6` for the routers; `victim`, `client2...client5`, `attacker` for the hosts; `rir` and `validator` for the RPKI containers) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does. A router line opens `vttysh`; the others open a plain shell.
- **Reset to baseline** withdraws both sides' announcements, removes the RPKI configuration and the ROA you created, and returns the lab to its converged Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup

Six ASes, each a single router running `vttysh`, each with one host behind it. AS1 holds `1.0.0.0/22` and originates it in BGP; its host answers at `1.0.0.1`. AS6 is the attacker. Two more containers form the RPKI infrastructure Part 2 uses. Every link in this topology is point-to-point: it connects exactly two routers and nothing else, so a router never has to work out which of several devices on a link a route came from.



Each node is labelled with the container name you pass to `docker exec`, and each end of a link with the address on that interface. The roles, and the host behind each AS:

AS	Role	Host
AS1	victim: originates <code>1.0.0.0/22</code>	victim (<code>1.0.0.1</code>)
AS2	transit: one hop from the victim	client2 (<code>2.0.0.100</code>)
AS3	transit: equidistant, and validates in Part 2	client3 (<code>3.0.0.100</code>)
AS4	transit: equidistant, symmetric to AS3	client4 (<code>4.0.0.100</code>)
AS5	transit: the attacker's only upstream, and the defender	client5 (<code>5.0.0.100</code>)
AS6	attacker: impostor served on <code>1.0.0.1</code>	attacker (<code>6.0.0.100</code>)

The AS-level links are AS1–AS2, AS2–AS3, AS2–AS4, AS3–AS5, AS4–AS5, and AS5–AS6. Counting hops along them gives the numbers the whole lab turns on:

AS-path length to	AS2	AS3	AS4	AS5
AS1 (victim)	1	2	2	3
AS6 (attacker)	3	2	2	1

AS2 is closer to the victim, AS5 is closer to the attacker, and AS3 and AS4 sit exactly between them.

Definition: Routing policy: route-map and local-preference

An operator can override BGP's default choice of route with policy, applied before AS-path length is ever compared. A *route-map* is a named, ordered list of rules a router applies to routes crossing one session: each rule matches on some property of a route and then permits or denies it. An 'empty permit' route-map, with no match clause at all, matches every route and denies none, so it has no effect; that is what every session in this lab starts with. *Local-preference* is a locally-set number a router prefers ahead of AS-path length entirely, the most common way an operator steers outbound traffic without touching what anyone announces.

No AS in this lab applies **local-preference** or any other policy. Every eBGP session already carries a **route-map**, but it is the empty permit rule described above, so it changes nothing. With policy out of the picture, only two things are left to decide a route in this lab: prefix length in the forwarding table, and AS-path length as BGP's tie-breaker once prefixes match exactly.

Router containers are <AS>_L3_BGP_router; host containers carry their role name, for example 5_L3_BGP_client and 6_L3_BGP_attacker. The RPKI containers are 0_L3_BGP_rir and 0_L3_BGP_validator.

Every device in this lab gives you one of two kinds of shell, and it matters which one you are in before you type anything:

- **Routers run BGP and nothing else.** A router shell opens **vttysh**, FRR's command-line, and everything you type there reads or changes BGP: the table of routes, the announcements this router sends, and the filters it applies. A router shell never carries the traffic the hijack actually diverts; for that you need a host.

```
# 6_L3_BGP_router: AS6's router, opened from your own terminal
docker exec -it 6_L3_BGP_router vtysh
```

- **Hosts are where you see the traffic itself.** A host shell opens a plain **bash** shell with **curl** and **traceroute** installed. The service at 1.0.0.1 answers every request with one line naming which AS is answering, so running **curl 1.0.0.1** from any host tells you, in plain text, whether that host's traffic still reaches the victim or has been diverted to the attacker.

```
# 5_L3_BGP_client: a client vantage in AS5, opened from your own terminal
docker exec -it 5_L3_BGP_client bash
```

Every command block in this handout opens with a comment naming the container the commands go in. Three vtysh commands carry most of the lab, and every router block below uses them:

```
# useful from here on: any router's vtysh session
# the whole table; > marks the best path for a prefix
show ip bgp
# one prefix in detail, with the reason its path won
show ip bgp 1.0.0.0/22
# enter configuration mode
conf t
```

One habit to carry through the rest of this handout: inside a router's **vttysh** shell, a comment always sits on its own line, above the command it explains, never at the end of a command line. **vttysh** does not treat **#** as a comment marker the way **bash** does; it reads everything after a command, including a trailing **#** ..., as part of the command itself, and rejects the whole line with **% Unknown command**. Every router block in this handout is already written this way, so you can select a whole block and paste it in one go without hitting this; keep the same habit if you ever type a router command of your own.

This lab gives you two ways to check what a route actually did, and they answer different questions. **show ip bgp**'s best-path origin says which AS's announcement a router picked as its route to a prefix: it tells you what the control plane decided, before any packet moves. The **curl** banner says which AS's service actually answered your request: it tells you what happened on the wire, after that decision was acted on. **scripts/status.sh** prints both for every AS at once, as a table of prefixes against ASes, and adds the RPKI state once Part 2 is under way.

Assessment. This lab is graded on nineteen questions, marked inline below as **Question N**, each placed at the step whose output it asks about. Answer all nineteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step.

Part 1: the attack

Definition: Convergence

The state where every router's table has settled: each router has picked its best route for every prefix it knows about, and nothing is still propagating because of a recent change. Convergence follows from every session first reaching **Established**, the point where two routers agree to exchange routes at all, but it is not the same event: Established only means the two routers are now talking, and routes still have to propagate and be re-evaluated at every hop after that before the network settles. This lab's **Status** can only report sessions as Established; it cannot show you the table settling directly, so treat Established as the signal to wait a little longer, not as convergence itself.

Read the converged network first

Spawn the lab and give it time to converge: watch **Status** until every router shows its eBGP sessions **Established**, then wait a few more seconds for the routes themselves to settle before you read anything. Before touching anything, read how traffic reaches the victim. On the AS5 router:

```
# 5_L3_BGP_router: AS5's router
docker exec -it 5_L3_BGP_router vtysh
show ip bgp
```

The table prints one line per path, and a prefix with two paths repeats the network column only on the first. Find the lines for 1.0.0.0/22:

```
# 5_L3_BGP_router prints this; it is output, not commands to type
  Network      Next Hop      Metric LocPrf Weight Path
*  1.0.0.0/22   10.0.45.1                0 4 2 1 i
*>              10.0.35.1                0 3 2 1 i
```

* marks a path as usable and > marks the one AS5 forwards on. The last column is the AS-path, read left to right from AS5's neighbour to the origin, so the chosen path crosses AS3 and AS2 to reach origin 1, the victim. Confirm it on the data plane from a client host:

```
# 5_L3_BGP_client: a client vantage in AS5
docker exec -it 5_L3_BGP_client bash
curl 1.0.0.1                # -> AS1: legitimate
traceroute 1.0.0.1          # the hops climb from AS5 up to AS1
```

Question 1. Read the AS-path to 1.0.0.0/22 on AS5 and on AS2, and write out both. Using the topology figure above, explain why one is three ASes long and the other is one.

Stage 1: originate the victim's own prefix

Definition: Prefix hijack

An AS originating a prefix it does not hold, so that BGP steers other ASes' traffic for that prefix to it. When the attacker originates the *same* prefix the real owner also originates, the two compete on AS-path length; this is also called a MOAS (multiple origin AS) hijack.

The attacker performs a **prefix hijack** by originating 1.0.0.0/22, the victim's own prefix, as if AS6 held it. On the AS6 router:

```
# 6_L3_BGP_router: AS6's router
docker exec -it 6_L3_BGP_router vtysh
conf t
# a matching route, so the announcement is stable
ip route 1.0.0.0/22 Null0
router bgp 6
 address-family ipv4 unicast
   # originate the victim's prefix as our own
   network 1.0.0.0/22
end
```

Question 2. Before you read the result, predict it: for each of AS2, AS3, AS4 and AS5, compare the AS-path length of the attacker's route with that of the legitimate route, and say which you expect to win. Then run `scripts/status.sh` and record which ASes actually changed to origin 6.

Question 3. Pick one AS whose best path changed to origin 6 and one whose did not. State the two AS-path lengths each of them was choosing between, and explain both outcomes from those lengths.

Definition: Best-path tie-break

When two routes to a prefix have equal AS-path length, BGP falls to further rules to pick one. The one that matters here is that a router keeps the route it already had rather than switching to a newly arrived one of the same length. FRR names the rule it used in `show ip bgp <prefix>`, on the line describing the chosen path.

AS3 and AS4 are the interesting case: they are two hops from the victim and two hops from the attacker, so the two routes tie. Read what AS3 did with that tie:

```
# 3_L3_BGP_router: AS3's router
docker exec -it 3_L3_BGP_router vtysh
# two paths; look at the one marked best
show ip bgp 1.0.0.0/22
```

Question 4. Write out both AS-paths AS3 is choosing between, state which one it kept, and quote the parenthesised reason FRR prints on the `best` line. Then say what AS3 would have done had the attacker's announcement arrived first, and what that implies about **tie-break** outcomes depending on the order of events.

Stage 2: originate a smaller piece of the prefix

Definition: More-specific (sub-prefix) hijack

Originating a more specific prefix that sits inside the victim's own prefix, for example 1.0.0.0/23 inside 1.0.0.0/22. Longest-prefix match (defined next) then picks the smaller announcement over the victim's for every address it covers, regardless of AS-path length; the victim's original, less specific prefix stays valid, it is just no longer used for the addresses the smaller one also covers. The victim continues to originate only the larger prefix.

Definition: Longest-prefix match

A router forwarding a packet uses the most specific matching route in its table, the one with the longest prefix length, regardless of how the route was learned or how long its AS-path is. A /23 entry beats a /22 entry for any address they both cover.

Run the **more-specific hijack**: withdraw the /22 and originate 1.0.0.0/23 instead.

```
# 6_L3_BGP_router: AS6's router
docker exec -it 6_L3_BGP_router vtysh
conf t
# the matching route the new announcement needs
ip route 1.0.0.0/23 Null0
router bgp 6
address-family ipv4 unicast
# stop originating the whole prefix
no network 1.0.0.0/22
# originate a more-specific slice of it instead
network 1.0.0.0/23
end
```

Read `show ip bgp` at each AS. A 1.0.0.0/23 line has appeared with origin 6, at AS2, AS3, AS4 and AS5 alike. The victim's 1.0.0.0/22 is untouched and still shows origin 1, and it is

no longer what anyone forwards on, because **longest-prefix match** picks the /23 for every address the two entries both cover. From every client host, `curl 1.0.0.1` now returns the attacker's banner.

Question 5. This time AS2 sends 1.0.0.1 traffic to the attacker, even though its AS-path to AS6 is three long and its AS-path to the victim is one. Explain why the longer path wins here when the same-length hijack in stage 1 could not take AS2.

Question 6. A packet arrives at AS2 destined for 1.0.0.1. AS2's table holds both 1.0.0.0/22 (origin AS1) and 1.0.0.0/23 (origin AS6). State which entry AS2 uses to forward it, why, and which AS the packet ends up at.

Question 7. During this hijack, run `curl 1.0.0.1` on the victim host itself (AS1): it still reaches the real service, while `client3` reaches the impostor. Explain why the victim is the one host the hijack cannot divert, in terms of the route each uses for 1.0.0.1.

Part 2: the defence

The victim's first instinct is to fight on the attacker's ground. If a more specific announcement wins, the victim can originate more specific prefixes of its own, so that its /23s match the attacker's /23 instead of losing to it as a /22. That works for exactly as long as the attacker has room to go smaller, and both sides run out of room at the same place: /24. The first half of this part walks that race to its end and measures what the victim has left.

The second half changes the question. Instead of arguing about which announcement is most specific, the victim publishes a signed record of which AS is allowed to originate its prefix, and routers check announcements against it. An announcement from the wrong AS is then wrong no matter how specific it is.

The deaggregation race

Definition: Deaggregation

Splitting a prefix into several smaller ones and originating those alongside it, so that the smaller announcements win longest-prefix match. An operator keeps originating the aggregate too, so networks that filter long prefixes can still reach it.

With the attacker's /23 still in place, have the victim **deaggregate**. AS1 splits its /22 into two /23s and originates both alongside the aggregate:

```
# 1_L3_BGP_router: AS1's router, the victim
docker exec -it 1_L3_BGP_router vtysh
conf t
ip route 1.0.0.0/23 Null0
ip route 1.0.2.0/23 Null0
router bgp 1
  address-family ipv4 unicast
    # the /22 aggregate stays announced alongside these two
    network 1.0.0.0/23
    network 1.0.2.0/23
  end
```

Now both sides originate 1.0.0.0/23, so longest-prefix match no longer separates them and the contest returns to AS-path length. Read the result with `scripts/status.sh`: AS2 goes back to origin 1, but AS3, AS4 and AS5 stay on origin 6.

Question 8. AS2 recovered and AS3 did not, although both now hear a 1.0.0.0/23 from each side. Explain each outcome from the AS-path lengths in the setup table, and quote the reason FRR prints on AS3's chosen path (`show ip bgp 1.0.0.0/23`) to support your answer for AS3.

The attacker answers by splitting too, withdrawing its /23 and originating the two /24s inside it:

```
# 6_L3_BGP_router: AS6's router
docker exec -it 6_L3_BGP_router vtysh
conf t
ip route 1.0.0.0/24 Null0
ip route 1.0.1.0/24 Null0
router bgp 6
 address-family ipv4 unicast
   # the /23 no longer beats the victim's /23s
   no network 1.0.0.0/23
   network 1.0.0.0/24
   network 1.0.1.0/24
end
```

Check both planes. The control plane, at any of the four transit ASes:

```
# 2_L3_BGP_router: AS2's router, then repeat on 3_, 4_ and 5_L3_BGP_router
docker exec -it 2_L3_BGP_router vtysh
# 1.0.0.0/24 and 1.0.1.0/24 are now in the table, both AS-paths ending in 6
show ip bgp
```

And the data plane, from your own terminal, without opening a shell on the client:

```
# 2_L3_BGP_client: one command, run from your own terminal
docker exec 2_L3_BGP_client curl -s 1.0.0.1
```

Every AS is back to origin 6 for those two /24s, and every client reads the impostor banner again.

Question 9. State the rule that lets the attacker's /24s beat the victim's /23s at every AS, including AS2, whose AS-path to the victim is shorter. Name what the victim would have to originate to win that comparison back.

The victim splits to /24s as well, dropping the /23s that can no longer win and originating all four /24s in their place:

```
# 1_L3_BGP_router: AS1's router, the victim
docker exec -it 1_L3_BGP_router vtysh
conf t
ip route 1.0.0.0/24 Null0
ip route 1.0.1.0/24 Null0
ip route 1.0.2.0/24 Null0
ip route 1.0.3.0/24 Null0
router bgp 1
address-family ipv4 unicast
  # a /23 cannot win against a /24
  no network 1.0.0.0/23
  no network 1.0.2.0/23
  network 1.0.0.0/24
  network 1.0.1.0/24
  network 1.0.2.0/24
  network 1.0.3.0/24
end
```

AS1 now originates five prefixes: the /22 aggregate it never withdrew, plus these four. Read what that bought it:

```
# 3_L3_BGP_router: AS3's router, then repeat on 2_, 4_ and 5_L3_BGP_router
docker exec -it 3_L3_BGP_router vtysh
# whose AS-path won, and the reason FRR gives
show ip bgp 1.0.0.0/24
```

This is the end of the ladder. Prefixes longer than /24 are filtered by most networks on the internet, so neither side can go further.

Question 10. Run `scripts/status.sh` and record, for 1.0.0.0/24, which ASes now use the victim's route and which still use the attacker's. Explain why the victim cannot recover the remaining ASes by deaggregating again.

Question 11. Count what each side had to originate at each rung of the ladder: the victim went from one prefix to three to five, the attacker from one to one to two. The victim is originating more than twice as many routes as the attacker to defend the same prefix, and still does not hold all of it. State what this says about deaggregation as a defence, and name the cost it imposes on every other network on the internet, not just on the victim.

Origin validation with RPKI

Definition: RPKI

The Resource Public Key Infrastructure: a set of certificates, issued by the registries that allocate address space, that prove which organisation holds which prefix. A certificate for 1.0.0.0/22 can only be issued to the holder of 1.0.0.0/22, so a statement signed under it cannot be forged by anyone else.

Definition: ROA and maxLength

A Route Origin Authorisation: a signed statement that a named AS may originate a named prefix. It carries a *maxLength*, the longest prefix the holder authorises. A ROA for 1.0.0.0/22 with maxLength 24 authorises the /22, both /23s and all four /24s. With no maxLength it authorises the /22 alone.

Definition: Relying party, VRP and RTR

A ROA sitting in a repository changes nothing by itself. A *relying party* (here Routinator, running in 0_L3_BGP_validator) fetches every repository, checks each signature back to the trust anchor, and reduces what survives to Validated ROA Payloads: plain (prefix, maxLength, origin AS) triples. It hands those to routers over the RPKI-to-Router protocol, RTR, on port 3323.

Definition: Route Origin Validation (ROV)

What a router does with the VRPs it holds. Each route is stamped **Valid** (a VRP covers the prefix and the origin AS matches), **Invalid** (a VRP covers the prefix but the origin AS or the length disagrees), or **NotFound** (no VRP covers it). `show ip bgp` prints the stamp as V, I or N in the first column. The stamp alone changes nothing: acting on it takes a route-map.

This lab runs its own **RPKI**. 0_L3_BGP_rir is a Krill certificate authority acting as the registry that allocated the address space, and it has already issued AS1 a certificate for 1.0.0.0/22 and AS6 a certificate for 6.0.0.0/24. AS6 has published a **ROA** for its own prefix; AS1 has published nothing, which is the gap you are about to close. A ROA on its own does not stop anything yet: nothing so far reads it. The next two steps only connect routers to the validator so they can see the ROAs that exist; nobody is enforcing anything until much further down.

Start by giving two routers a validator to talk to. FRR's own commands and error messages below call this connection an RPKI *cache*, but it is the same relying party defined above; AS5 is the attacker's upstream, and AS3 is a bystander that will only watch:

```
# 3_L3_BGP_router: AS3's router
docker exec -it 3_L3_BGP_router vtysh
conf t
rpki
rpki polling_period 10
rpki cache 172.28.3.1 3323 preference 1
exit
end
# expect: (connected)
show rpki cache-connection
# expect one entry, 6.0.0.0/24 from AS6
show rpki prefix-table
```

Run the two `show` commands a couple of seconds after `end`. Immediately after leaving configuration mode both still report `No connection to RPKI cache server`, because the router has not finished opening the RTR session yet; that is not a misconfiguration.

The same on AS5, which reaches its own validator interface at 172.28.5.1:

```
# 5_L3_BGP_router: AS5's router
docker exec -it 5_L3_BGP_router vtysh
conf t
rpki
rpki polling_period 10
rpki cache 172.28.5.1 3323 preference 1
exit
end
show rpki cache-connection
```

Now look at what AS3 makes of the network with only AS6's ROA published:

```
# 3_L3_BGP_router: AS3's router
docker exec -it 3_L3_BGP_router vtysh
# V, I or N in the first column
show ip bgp
```

Question 12. With only AS6's ROA in place, state the validation stamp AS3 gives to 6.0.0.0/24 and the stamp it gives to the attacker's 1.0.0.0/24. Explain why the hijacked route does not come out Invalid at this point.

So far only a stamp exists: nothing anywhere in this lab is refusing a route because of it. That is deliberate, and it matches most of the real internet: more than half of the internet's address space carries a ROA today, but comparatively few networks enforce anything on top of one. Now bring in the rarer piece, on AS5 only. The **ROV** route-map refuses Invalid routes and passes everything else:

```
# 5_L3_BGP_router: AS5's router
docker exec -it 5_L3_BGP_router vtysh
conf t
route-map ROV deny 10
match rpki invalid
exit
route-map ROV permit 20
exit
router bgp 5
address-family ipv4 unicast
neighbor 10.0.56.2 route-map ROV in
end
```

Nothing changes: every client still reads the impostor banner. That is the correct outcome: the route-map can only refuse a route the validator has actually marked Invalid, and nothing is marked Invalid yet, because no ROA covers the victim's prefix.

Question 13. Explain why `route-map ROV` drops nothing at this point, in terms of the stamp AS5 is giving the attacker's routes. Then say what would break if the route-map denied Not-Found routes as well as Invalid ones, given how many prefixes on the real internet have no ROA.

The victim registers a ROA

Open the registry portal at <https://127.0.0.1:3000> and sign in with the token `minilabs-rpki-token`. Your browser will warn about the certificate, which is expected:

the lab's registry signs its own. Under the AS1 certificate authority, add a ROA for 1.0.0.0/22 originated by AS 1, and leave the max length field empty for now.

The same thing from the command line, if you prefer it:

```
# 0_L3_BGP_rir: the registry's own command line, driven from your terminal
docker exec -it 0_L3_BGP_rir krillc roas update --ca AS1 --add "1.0.0.0/22 => 1"
docker exec -it 0_L3_BGP_rir krillc roas list --ca AS1
```

The listing reads back 1.0.0.0/22-22 => 1: leaving the max length unset does not mean “any length”, it means the ROA authorises the /22 and only the /22.

The ROA has to travel before any router sees it: Krill publishes it, the validator re-checks its repository on a 20-second timer, and each router polls the validator every 10 seconds. Allow about 40 seconds, then read AS3 again:

```
# 3_L3_BGP_router: AS3's router
docker exec -it 3_L3_BGP_router vtysh
clear bgp ipv4 unicast * soft in
show ip bgp
```

Every /24 in the victim's prefix is now marked I, including 1.0.2.0/24 and 1.0.3.0/24, which the attacker never touched and which AS3 learned from the victim through AS2.

Question 14. The victim has just marked its own announcements Invalid. Explain, from the ROA you registered and the **maxLength** rule, why 1.0.2.0/24 carrying AS-path 2 1 comes out Invalid. Then state what would happen to the victim's traffic if AS2, AS3 and AS4 all enforced ROV the way AS5 now does.

Fix the ROA by giving it a max length of 24, either in the portal or from the command line:

```
# 0_L3_BGP_rir: the registry's own command line, driven from your terminal
docker exec -it 0_L3_BGP_rir krillc roas update --ca AS1 \
    --remove "1.0.0.0/22 => 1" --add "1.0.0.0/22-24 => 1"
```

Question 15. After the change reaches AS3, record the stamp on 1.0.2.0/24 (AS-path 2 1) and on 1.0.0.0/24 (AS-path 5 6). Explain what maxLength 24 authorises that maxLength 22 did not, and why setting it to 32 instead would be a mistake.

Validating is not enforcing

Look at AS3's entry for the hijacked prefix now:

```
# 3_L3_BGP_router: AS3's router
docker exec -it 3_L3_BGP_router vtysh
show ip bgp 1.0.0.0/24
```

AS3 reports `rpki validation-state: invalid` on the attacker's path and still marks that path `best`. On `client3`, `curl 1.0.0.1` still returns the impostor.

Question 16. AS3 knows the route is Invalid and is using it anyway. Explain what AS3 is missing that AS5 has, and state what an operator gains from running a validator without any

route-map at all.

Now look at AS5, which does have the route-map. It has been in place since before the ROA existed:

```
# 5_L3_BGP_router: AS5's router
docker exec -it 5_L3_BGP_router vtysh
show ip bgp 1.0.0.0/24
```

AS5 also reports the attacker's path as `invalid`, and is also still using it. The route-map says to deny Invalid routes, and the route is Invalid, and it is still there. This is not a misconfiguration: FRR evaluates a route against an inbound route-map when the route is first received, and does not re-run that evaluation just because the route's RPKI validation state changes afterwards underneath it. AS5 received this route before the ROA existed, when it was still `NotFound` and so passed the route-map's `permit 20` clause; the stamp has since changed, but nothing has told AS5 to look again. `clear bgp ipv4 unicast * soft in` asks AS5's neighbours to resend their routes and runs the route-map over them again from scratch.

Question 17. Predict what `show ip bgp 1.0.0.0/24` on AS5 will show once you run the command below, then run it and check.

Run that command on AS5:

```
# 5_L3_BGP_router: in the vtysh session you already have open
clear bgp ipv4 unicast * soft in
```

The hijacked routes leave AS5's table. Because AS5 is the only path out of AS6, they never reach AS3, AS4 or AS2 either, and every client's `curl 1.0.0.1` returns `AS1: legitimate` again.

Question 18. AS5 is the only AS enforcing ROV, yet AS2, AS3 and AS4 all recovered. Explain the propagation that produces that, then state what would still be hijacked if the attacker had a second upstream that did not enforce ROV.

Question 19. Compare the two defences you have run. Deaggregation topped out at five simultaneous announcements and still could not win back AS3, AS4 or AS5, because /24 is as far as either side can deaggregate; ROV needed only a signed ROA and a route-map on somebody else's router, and recovered all three. Name one property of the ROA that the deaggregation approach can never have, and one thing an attacker can still do to a prefix that has a correct ROA and is validated everywhere.

Checks: what “done” means

- **Attack works:** with no defence, the /22 hijack makes AS5's best-path origin for 1.0.0.0/22 become 6 and `client5`'s `curl` return the attacker; the /23 hijack makes 1.0.0.0/23 origin 6 at AS2 through AS5, and every client's `curl` returns the attacker.
- **Deaggregation is not enough:** with both sides originating /24s, AS2 reads the victim's banner again but `client3`, `client4` and `client5` still read the impostor.
- **Defence holds:** with the ROA registered at maxLength 24 and the ROV route-map on AS5's session to AS6, no hijacked prefix appears at any AS and every client's `curl 1.0.0.1` returns `AS1: legitimate`.

`scripts/status.sh` prints the established sessions, a table of best-path origins for every prefix in play, what the registry has signed, what the validator serves, and each client's `curl`, so you can read before and after without shelling in. To undo your work, `scripts/reset.sh` withdraws both sides' announcements, removes the RPKI configuration and the ROA you created, and returns to the converged Part 1 baseline.

Safety. All six ASes live on isolated point-to-point links with no route off the host. The hijack announcements and the diverted traffic only ever reach the lab's own containers; no BGP leaves your machine. The RPKI containers share a private Docker bridge so that the registry portal can be reached from a browser on this machine, and that portal is bound to `127.0.0.1`, so it is not reachable from the network. The lab's registry is its own trust anchor and is never loaded with a real one, so nothing here can affect or be affected by the real RPKI.

When you are done, tear the lab down. Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left running on your host. **Reset to baseline** only rewinds the attack and the defence; only **Teardown** deletes the lab.